

## Trabalhando em equipes usando o GitHub

Este roteiro procura apresentar duas formas de se trabalhar em equipe usando o GitHub. No primeiro todos os integrantes da equipe têm os mesmos direitos de acesso. No segundo existe a figura de um mediador que precisa autorizar as contribuições dos demais integrantes da equipe.

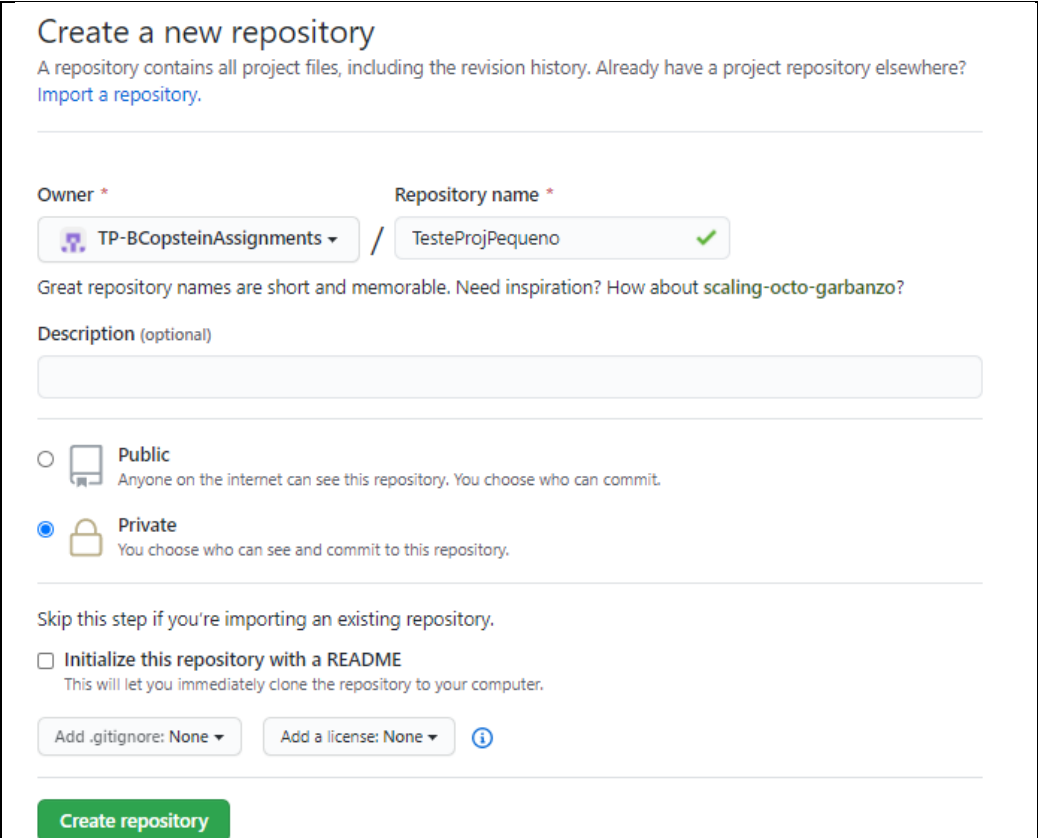
A primeira forma está descrita em mais detalhes do que a segunda. Ambas estão descritas na seção 5.1 do livro [ProGit](#).

### Acesso liberado a todos os participantes

Neste roteiro iremos imaginar uma equipe de 3 pessoas Luis, Suzana e Jorge. Luis será o “líder” do projeto. Ele é que irá criar o repositório inicial.

1. Criação do repositório original: Luís já dispõe do início do projeto em seu computador pessoal. Então cria o repositório do projeto no GitHub e acrescenta Suzana e Jorge na equipe. Todos têm direitos de administrador sobre o projeto.

#### a. Criação do repositório



**Create a new repository**

A repository contains all project files, including the revision history. Already have a project repository elsewhere? [Import a repository.](#)

---

**Owner \*** **Repository name \***

 TP-BCopsteinAssignments / TesteProjPequeno 

Great repository names are short and memorable. Need inspiration? How about [scaling-octo-garbanzo?](#)

**Description (optional)**

---

☐  **Public**  
Anyone on the internet can see this repository. You choose who can commit.

☒  **Private**  
You choose who can see and commit to this repository.

---

Skip this step if you're importing an existing repository.

☐ **Initialize this repository with a README**  
This will let you immediately clone the repository to your computer.



---

**Create repository**

#### b. Repositório criado: botão para acrescentar a equipe

**Give access to the people you work with**  
You should give access to the collaborators and teams you need to work with.

[Add teams and collaborators](#)

**Quick setup — if you've done this kind of thing before**

[Set up in Desktop](#) or [HTTPS](#) [SSH](#) <https://github.com/TP-BCopsteinAssignments/TesteProjPequeno>

Get started by [creating a new file](#) or [uploading an existing file](#). We recommend every repository include a [README](#)

**...or create a new repository on the command line**

```
echo "# TesteProjPequeno" >> README.md
git init
git add README.md
git commit -m "first commit"
git remote add origin https://github.com/TP-BCopsteinAssignments/TesteProjPequeno.git
git push -u origin master
```

### c. Criação da equipe

Options

**Manage access**

Security & analysis

Webhooks

Notifications

Integrations

Deploy keys

Secrets

Actions

**Who has access**

**PRIVATE REPOSITORY**   
Only those with access to this repository can view it.  
[Manage](#)

**BASE ROLE** [Read](#)  
All 1 members can access this repository.  
[Manage](#)

**DIRECT ACCESS**  
0 teams or members have access to this repository. Only **Owners** can manage access to this repository.

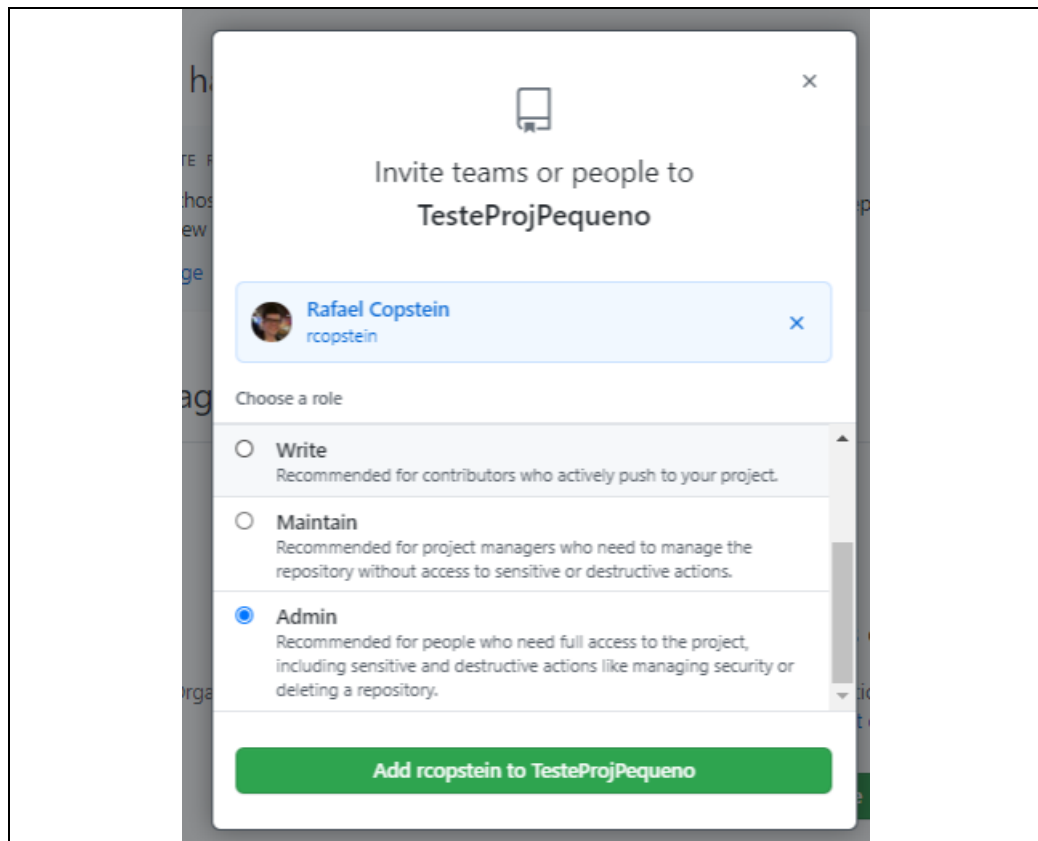
**Manage access**



**You haven't added any teams or people yet**

Organization owners can manage individual and team access to the organization's repositories. Team maintainers can also manage team's repository access. [Learn more about organization access](#)

[Invite teams or people](#)



2. Vinculação do projeto local com o repositório remoto: uma vez que o repositório no GitHub está corretamente configurado, Luis vincula seu projeto local ao repositório remoto e sincroniza seu conteúdo. O endereço web que aparece abaixo (em vermelho) é o do repositório criado no GitHub.

```
git remote add origin https://github.com/Testes/TesteProjPequeno.git  
git push -u origin master
```

A partir de agora o repositório pode ser compartilhado pelos 3 membros da equipe.

3. Clonando o repositório: tanto Suzana como Jorge podem agora “clonar” o projeto para suas máquinas locais. Para tanto usam o comando “git clone <https://github.com/Testes/TesteProjPequeno.git>”. Esse comando irá copiar não apenas os arquivos do projeto, mas também o diretório “.git” que possui todo o histórico do projeto e a vinculação com o repositório remoto. A partir desse momento todos podem fazer suas contribuições ao projeto.
4. Criando uma branch para cada atividade: em princípio cada desenvolvedor pode começar a trabalhar nas suas próprias classes, fazer seus “commits” e quando necessário fazer “push” para integrar seu código no repositório. Vamos imaginar, porém, que Suzana está trabalhando em uma classe. A classe ainda não está pronta, o código não compila, mas ela precisa ir para a aula e irá terminar seu trabalho no dia seguinte. Ela quer salvar o estado atual do seu trabalho no repositório para não correr o risco de perder nada, mas se fizer isso irá subir código que não compila prejudicando os colegas que, eventualmente, sincronizem o repositório (não conseguiram mais

compilar o código). Para evitar problemas desse tipo costuma-se adotar uma “branch” para cada atividade. Ao iniciar a criação da classe “X”, Suzana cria uma branch chamada “CriaX” passando a desenvolver nessa branch. Como se trata de uma branch separada da “master” pode fazer quantos “commits” e “push” quiser porque apenas sua branch será afetada. Quando sua classe estiver concluída ela pode então fazer um “merge” da sua branch com a branch “master”.

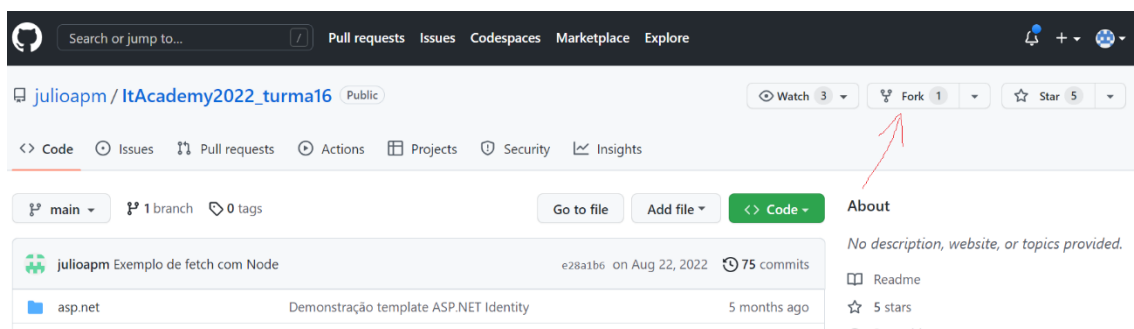
5. Sincronizando com o repositório principal: eventualmente quando for fazer um “push” para enviar suas contribuições para o repositório principal o Git irá reclamar que o repositório de Suzana está “atrasado” em relação ao principal, isto é, novas contribuições já foram feitas em relação ao conteúdo que ela tem na máquina local. Uma maneira simples de resolver isso é solicitar que GitHub baixe a versão atual integrando-a com o repositório local. Isso pode ser feito através do comando “git pull” (que corresponde a um “fetch” seguido de um “merge”). Uma vez que o repositório local está atualizado é possível fazer um “push”.

O processo descrito é bem simples, pressupõe um certo nível de interação entre os participantes de maneira a evitar muitos conflitos (uma forma simples é cada um trabalhar apenas em classes pré-definidas pelo líder), mas funciona para pequenos projetos tais como trabalhos de aula e é simples de ser compreendido e executado por iniciantes.

### Acesso controlado por mediador

Nesta segunda forma de trabalho o mediador cria o projeto público e não acrescenta ninguém na equipe.

Como os demais integrantes da equipe só tem direito de leitura, não podem subir modificações para o projeto. Então ao invés de “clonarem” o projeto eles devem fazer um “fork” do mesmo (veja na figura como criar um “fork” de um projeto).



Quando se “clona” um projeto cria-se uma cópia do projeto na máquina local vinculado ao projeto na nuvem. Então, se tivermos autorização de escrita no projeto original, basta fazer um “push” das modificações para que elas sejam integradas ao projeto.

O “fork” funciona diferente. Ele cria uma cópia do projeto na conta na nuvem de quem solicitou o “fork”. Esta cópia passa a ser de propriedade desta pessoa. Então ela pode “clonar” o projeto e fazer as alterações que quiser. Quando ela quiser integrar as modificações com o projeto original deverá solicitar um “pull-request”. O “pull-request” envia uma notificação de integração para o dono do projeto original que pode optar por aceitar ela ou não. Desta forma todas as contribuições para o projeto têm de ser aceitas pelo mediador.

